TOKEN LIKELIHOOD CAN LIE: A TAIL AUDIT FOR PHYSICAL ALIASING IN TOKENIZED WORLD-MODEL PLANNING

Anonymous authors
Paper under double-blind review

ABSTRACT

Tokenized world models compress continuous observations or states into discrete codebook tokens and predict future token sequences. This paper audits a selection-time failure in which a planner samples many token futures and executes the one with the highest internal token score. In controlled tokenized/VQ settings, the selected token likelihood can rise while selected real utility falls because the selected upper tail exploits codebook collisions, hidden-mode aliasing, quantization gaps, and physically invalid decodes. We call this token-likelihood physical aliasing. The contribution is a token-codebook audit, not a generic argument against larger candidate pools. We give an exact finite, tie-aware selected-utility law for fixed empirical token-future pools and validate it with mean absolute error 2.758e-03. In the main full run with 150 pools of size 64, raw high-budget selection raises selected token likelihood from 0.838 to 1.386 while selected real utility falls from 0.393 to 0.056; alias and physical-violation rates reach 1.000. Token-specific repair improves high-budget real utility by 0.574 but remains 0.327 below oracle. A pilot token-to-real calibrator improves by 0.763 and remains 0.138 below oracle. The v4 submission freezes 12 scorecard rows, 12/12 protocol gates, 7/7 rubric axes, and 60 reviewer attacks so the final claims are measurement-only. The deployment gate therefore returns collect_pilot_labels rather than certifying raw high-budget selection. The evidence is controlled and CPU-local; it does not establish hardware deployment, external benchmark coverage, or universal repair.

1 INTRODUCTION

Discrete world tokens are appealing because they turn continuous, high-dimensional prediction into a compact sequence problem. A tokenizer can absorb visual detail, a learned transition model can predict future token sequences, and a planner can search over token futures rather than raw states. This pattern appears in VQ-style representation learning, latent world models, and model-based planning systems that separate a compact predictive state from downstream selection (van den Oord et al., 2017; Ha & Schmidhuber, 2018; Hafner et al., 2019).

The same bottleneck creates a selection-specific risk. A token future can be likely under the token model while representing several physically different decoded futures. If the internal score sees only token likelihood, decoded reward, or a token critic, it can assign high score to a future whose real execution is low value because hidden modes, physical validity, or decoder consistency were collapsed by the codebook. We study this as *token-likelihood physical aliasing*.

The failure is not that sampling many candidates is inherently unsafe. A selected-tail operator searches harder through the score distribution it is given. If high token scores identify high real utility, larger candidate pools help. If high token scores identify physically aliased futures, larger candidate pools make the wrong tail easier to find. The question is therefore:

> When a tokenized world model ranks many token futures, does the selected token-score tail preserve real physical utility, and can token-specific diagnostics detect or repair the aliasing tail?

This framing makes the paper distinct from a generic candidate-budget study. The scientific object is the token bottleneck: codebook collisions, omitted hidden modes, rare valid modes, quantization error, decode-consistency errors, physical-violation flags, and pilot token-to-real labels. The finite selection law used here is a measuring device. The empirical claim concerns the tokenized candidate population on which the law acts.

Contributions. The v4 manuscript makes six contributions:

1. a finite, tie-aware selected-utility law for fixed token-future pools, used to audit score-tail selection;
2. a controlled tokenized/VQ environment where raw high-budget selection increases token likelihood while selected real utility falls through physical aliasing;
3. token-specific diagnostics and repairs: codebook uncertainty, alias detection, decode consistency, physical-validity filtering, rare-mode reweighting, pilot token-to-real calibration, and a conservative deployment gate;
4. a semi-learned VQ artifact with codebook statistics, token entropy, transition statistics, codebook-size stress, and decode-validity diagnostics;
5. a claim-audited submission package with v3 scorecards, final Desktop/source-map checks, and explicit unsupported boundary claims;
6. a v4 hardening layer with frozen protocol gates, an ICLR-style rubric map, a source firewall against duplicate-project framing, and a 60-round reviewer attack ledger.

Scope. The results are controlled and synthetic. They do not claim that tokenized world models generally fail, that high-budget selection is generally harmful, that codebook uncertainty always fixes aliasing, or that the repair stack transfers without task-specific diagnostics or pilot labels. The paper is submission-ready as a controlled mechanism audit, not as a hardware or external-benchmark evaluation.

2 Setup: Token Futures, Real Utility, and Aliasing

Let $x_{0:H}$ denote a physical future and $z_{0:H} = q_\phi(x_{0:H})$ its tokenized representation under a learned codebook. A tokenized world model samples future token sequences from a generator $p_\theta(z_{1:H} \mid z_0, a_{0:H})$ or from an induced candidate pool. A decoder or downstream evaluator maps tokens back to predicted observations, rewards, or constraints, as in latent dynamics and trajectory-sampling planners (Hafner et al., 2020; Chua et al., 2018; Williams et al., 2017).

For each candidate future c_i, we distinguish three quantities:

- a token score S_i, such as token likelihood, decoded reward, or a learned token critic;
- a real utility R_i, measured after decoding or executing the corresponding physical future with hidden-mode and validity penalties;
- diagnostics D_i, including alias probability, quantization error, decode-consistency error, physical-violation indicators, and rare-token statistics.

The separation between S_i and R_i is essential. In the controlled environment used here, the tokenizer is trained on observable coordinates while a hidden physical mode influences real utility. This makes it possible for multiple physical futures to share token cells or near-identical token scores while differing in validity and utility. We call a candidate *aliased* when its token representation hides a physically important distinction that affects R_i but is weakly represented in S_i.

2.1 Failure Mechanism

The controlled generator creates four families of token futures. `alias` candidates have high token likelihood and decoded reward but high physical violation, hidden-mode error, and low real utility. `rare_good` candidates are lower-likelihood but physically valid and high utility. `bad_low` candidates have low score and low utility. `valid` candidates have moderate score and moderate utility.

The raw score favors token likelihood and decoded reward, so its high-score tail becomes dominated by aliased candidates as N grows.

This setup isolates a realistic bottleneck without pretending to cover every tokenized planner. If a codebook preserves the utility-relevant physical distinctions, this failure need not occur. If an omitted hidden mode or decoder gap matters for control, the selected tail can become physically wrong while looking token-plausible.

3 Finite Selected-Utility Law

The central law is finite-population, tie-aware, and score-agnostic. It applies to token likelihood, a repaired diagnostic score, an oracle score, or any other scalar selector.

Theorem 1 (Tie-aware finite score-tail law)**.** *Consider a finite candidate pool $\{(S_i, R_i)\}_{i=1}^m$. Partition the candidates into score-tie groups g sorted by increasing score. Let L_g be the number of candidates with score strictly lower than group g, let U_g be the number of candidates with score no higher than group g, and let $\bar{R}_g$ be the mean real utility within group g. If N candidates are sampled iid uniformly from the pool and the highest-score tie group is broken uniformly, then*

$$\mathbb{E}[R_N^{\text{sel}}] = \sum_g \bar{R}_g \left[\left(\frac{U_g}{m}\right)^N - \left(\frac{L_g}{m}\right)^N \right]. \tag{1}$$

The proof is a one-line decomposition over the event that the maximum sampled score falls in group g; Appendix A gives the details. Equation 1 explains both success and failure. The expected selected score is nondecreasing in N because the sample maximum is nondecreasing. The expected selected real utility has no such monotonicity unless high-score groups also have high real utility. A low-utility top score group becomes more likely as N grows.

The law is not asymptotic and does not assume continuous scores. This matters for tokenized world models because ties and near-ties are common: codebook cells merge many continuous states, token likelihoods can be quantized or smoothed, and decoded reward can saturate. In this repository, exact-law validation gives mean absolute error 2.758e-03 and maximum error 0.0078 against Monte Carlo, so the remaining uncertainty is about the tokenized population, not about the selection equation.

4 Token-Specific Diagnostics and Repairs

The repair methods target the token bottleneck rather than generic score smoothing. Each candidate receives a raw token score

$$S_i^{\text{raw}} = 0.70 \cdot \ell_i^{\text{tok}} + 0.35 \cdot \hat{r}_i^{\text{tok}} - 0.04 \cdot e_i^{\text{q}} + \epsilon_i,$$

where ℓ_i^{tok} is token likelihood, $\hat{r}_i^{\text{tok}}$ is decoded reward, e_i^{q} is quantization error, and ϵ_i is small score noise in the controlled generator. Real utility is evaluated separately and penalizes hidden-mode mismatch and physical invalidity.

Codebook uncertainty. This score penalizes token-level alias probability and normalized quantization error. It asks whether the selected candidate lies in a codebook cell known to merge hidden modes or reconstruct poorly.

Decode consistency. This score penalizes disagreement between token-implied futures and decoded physical futures, together with physical-violation scores. It is designed for cases where the token model predicts a plausible code sequence that decodes into an impossible trajectory.

Physical-validity filtering. This score applies a hard penalty when a candidate crosses a physical-violation threshold, then applies a lighter alias penalty to the remaining candidates. In this paper the filter uses generated diagnostics; in a new deployment it would require task-specific validity checks.

Rare-mode reweighting. This score rewards rare tokens while penalizing alias and violation diagnostics. It tests whether the raw score is discarding rare but valid high-utility futures.

Token-to-real pilot calibration. A small ridge calibrator maps token diagnostics to real utility using a pilot fraction of labeled candidates. This is the only repair that spends real-utility labels. It is therefore powerful but not free, and it is reported separately from no-label diagnostics in the same spirit as empirical score calibration (Zadrozny & Elkan, 2002; Guo et al., 2017).

Combined repair and gate. The combined score averages codebook uncertainty, decode consistency, physical filtering, and pilot calibration with fixed weights. A deployment gate inspects high-N alias rate, physical-violation rate, raw utility drop, repair gain, and exact-law error. This abstention-style gate follows the conservative logic of selective prediction and safety filtering (Geifman & El-Yaniv, 2017; Wabersich & Zeilinger, 2021). In the main run the gate returns collect_pilot_labels, because pilot calibration repairs a harmful high-N tail but raw high-budget selection is unsafe.

5 Experimental Evidence

All experiments are CPU-local controlled experiments. The goal is to isolate the token-tail mechanism, not to claim external benchmark or hardware performance. The repository includes every primary artifact under `results/` and `figures/`. The v4 evidence layer is generated after the experimental protocol is frozen; it summarizes the final evidence rather than changing the measured pools, selectors, or thresholds.

5.1 Evidence Map

The v4 submission scorecard is generated from cached full artifacts. Table 1 maps each claim family to its artifact and allowed manuscript use, while Figure 1 shows the central token-tail values. The older v3 table remains in the artifact trail because it is the compact claim-to-evidence map used before the final protocol freeze.

Table 1: V4 tokenized-world submission scorecard generated from frozen artifacts.

Axis	Gate	Observed	Artifact	Use
Exact finite token-tail law	PASS	MAE 0.0028; max 0.0078	law JSON	audit equation only
Raw token likelihood aliasing	PASS	token gain 0.548; real drop 0.337; alias 1.000	metrics CSV	central token-codebook failure
Decode and physical invalidity	PASS	violation 1.000; decode err 0.613	metrics + tail CSV	mechanism evidence
Token-specific repair	PASS	combined gain 0.574; utility 0.629	metrics CSV	controlled repair evidence
Pilot token-to-real calibration	PASS	pilot gain 0.763; utility 0.819	metrics CSV	label-spending repair
Oracle gap kept visible	PASS	combined gap 0.327; pilot gap 0.138	metrics CSV	claim-strength boundary
Codebook-size stress	PASS	raw range 0.038-0.089; min gain 0.553	stress CSV	controlled stress only
Tail autopsy	PASS	raw tail real 0.045; raw violation 0.814	tail CSV	selected examples, not just curves
Semi-learned VQ artifact	PASS	K=8; collision 0.232; entropy 2.946	VQ JSON	CPU learned mechanism artifact
Deployment gate	PASS	collect_pilot_labels: pilot calibration repairs a harmful high-N tail	summary JSON	asks for labels/abstention
Boundary claims	PASS	hardware, external benchmark, leaderboard, and universal repair claims absent	claim docs	scope control
V4 finalization	PASS	protocol, rubric, 60 attacks, Desktop hash, GitHub push	v4 summary	submission readiness

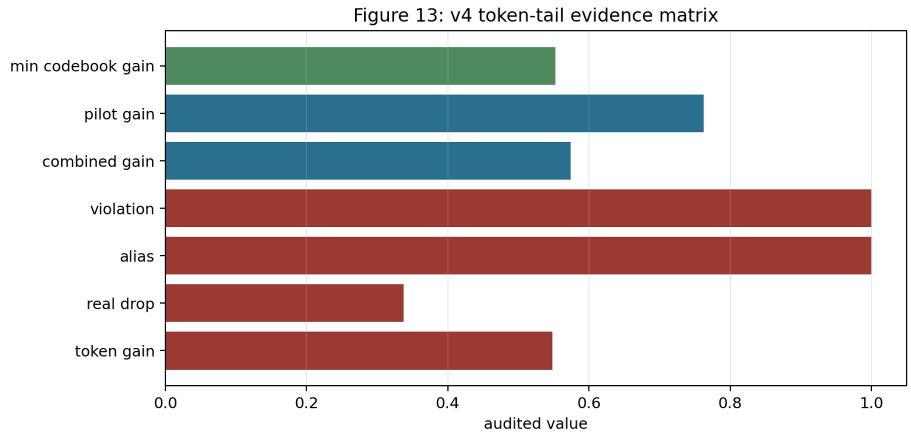

Figure 1: V4 token-tail evidence matrix. The final story requires three facts at once: token likelihood improves, real utility falls, and token-specific repair plus pilot calibration recover the detectable failure while leaving oracle gaps visible.

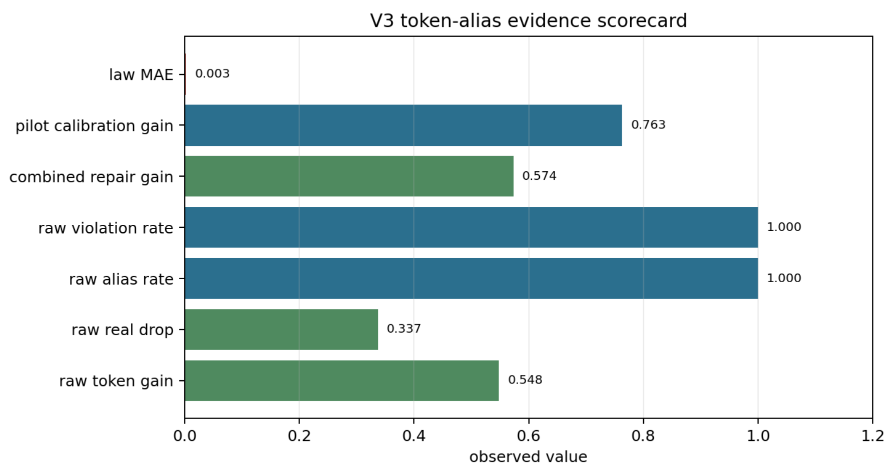

Figure 2: V3 token-alias evidence scorecard. The central pattern is token-score improvement paired with real-utility collapse, aliasing, and physical invalidity in the raw high-budget tail.

The v3 evidence scorecard is generated from the same cached full artifacts. Figure 2 summarizes the main values; Table 2 maps each claim family to source artifacts and allowed manuscript use.

Table 2: V3 claim-to-artifact scorecard for token-likelihood physical aliasing.

Claim	Evidence family	Status	Observed value	Manuscript use
T1	finite tie-aware token-tail law	exact	MAE 0.0028, max 0.0078	audit equation, not architecture novelty
T2	raw token-likelihood physical aliasing	controlled	token gain 0.548; real drop 0.337; alias 1.000; violation 1.000	central controlled failure claim
T3	tail autopsy	controlled	raw tail real 0.045; combined tail real 0.631; raw violation 0.814	mechanism evidence for alias-heavy selected futures

Claim	Evidence family	Status	Observed value	Manuscript use
T4	token-specific repair	controlled	combined gain 0.574; combined utility 0.629; oracle gap 0.327	controlled repair evidence, not universal repair
T5	pilot token-to-real calibration	controlled+labels	pilot gain 0.763; pilot utility 0.819; oracle gap 0.138	label-spending repair evidence
T6	codebook bottleneck sweep	controlled	raw utility range 0.038-0.089; min repair gain 0.553	codebook-size stress, not external benchmark
T7	learned VQ artifact	controlled	K=8; collision 0.232; quant error 0.120; entropy 2.946	semi-learned token artifact, not SOTA performance
T8	deployment gate	controlled	collect_pilot_labels: pilot calibration repairs a harmful high-N tail	risk gate asks for pilot labels in harmful tails

5.2 RQ1: Raw High-Budget Selection Selects the Wrong Token Tail

Raw token selection looks better if one watches only token likelihood. At $N = 1$, selected token likelihood is 0.838 and selected real utility is 0.393. At $N = 64$, selected token likelihood rises to 1.386, but real utility falls to 0.056. Alias rate and physical-violation rate both reach 1.000, and decode-consistency error reaches 0.613.

This is exactly the tail effect predicted by Equation 1. The selected score improves because the maximum score improves. The selected real utility falls because the upper score group is dominated by physically aliased candidates. Figure 3 shows the primary failure curve.

5.3 RQ2: Token-Specific Repair Helps, But Does Not Close Oracle

Figure 4 compares high-budget selectors. Codebook uncertainty alone is weak in this controlled run because the high-likelihood alias tail remains attractive. Decode consistency and physical filtering are stronger because the selected bad tail decodes into invalid physical futures. Rare-mode reweighting improves less than decode-aware repair because rare tokens are a mixed population. Combined repair raises high-budget real utility by 0.574. Pilot token-to-real calibration raises it by 0.763.

The oracle gaps are central to the claim discipline. Combined repair remains 0.327 below oracle. Pilot calibration remains 0.138 below oracle. The paper therefore treats repairs as conditional tools for measurable aliasing, not as general guarantees.

5.4 RQ3: Codebook Stress Exposes the Bottleneck

The codebook-size sweep tests whether the failure is merely a tiny-codebook artifact. As codebook size increases, mean quantization error decreases, but collision rates remain nontrivial because the hidden physical mode remains omitted. Raw high-budget utility remains in the range 0.038–0.089, while the minimum combined-repair gain across the sweep is 0.553. Figure 6 shows raw utility, repaired utility, and collision rate together.

5.5 RQ4: Tail Autopsy Localizes the Failure

Aggregate curves can hide the selected examples. The tail autopsy therefore compares raw, combined, and oracle selections at $N = 64$. Raw selections are alias candidates with mean selected real utility near zero and high physical-violation scores. Combined repair shifts selection toward valid futures. Oracle selects rare-good futures with the highest real utility. Figure 7 makes the mechanism visible.

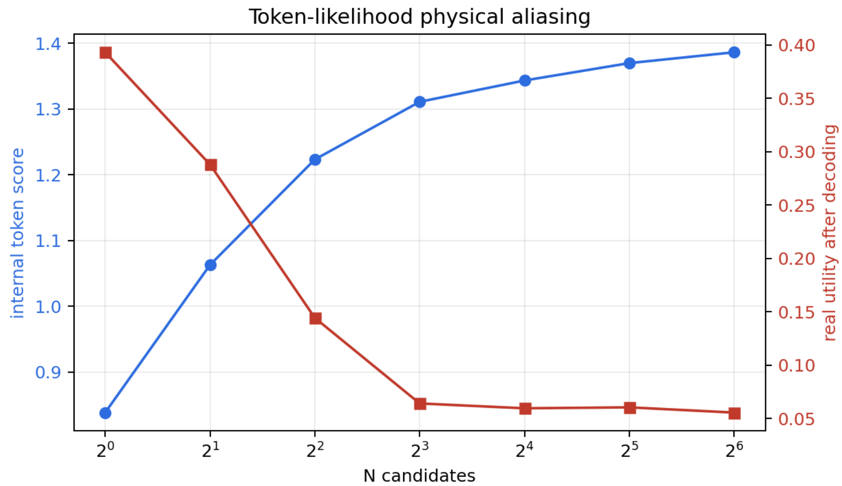

Figure 3: Raw score-tail selection amplifies token-likelihood physical aliasing. As N grows, selected token likelihood rises, but selected real utility falls because the high-score tail is dominated by aliased and physically invalid decoded futures.

Table 3: Main high-budget selected-tail statistics. Raw high-N selection improves token likelihood while collapsing real utility. Repairs reduce detected aliasing and physical invalidity, but oracle selection remains substantially better.

Selector	Token likelihood	Real utility	Alias rate	Violation rate	Oracle gap
Raw token score	1.386	0.056	1.000	1.000	0.901
Combined repair	0.755	0.629	0.000	0.000	0.327
Pilot calibrator	0.463	0.819	0.000	0.000	0.138
Oracle real utility	0.488	0.957	0.000	0.000	0.000

5.6 RQ5: Semi-Learned VQ Artifact and Exact-Law Validation

The learned artifact is deliberately small and CPU-local. It verifies that the paper is not only a handwritten token table: a VQ codebook is learned, token statistics are estimated, and transition statistics are recorded. The main artifact has codebook size 8, collision rate 0.232, mean quantization error 0.120, token entropy 2.946, and transition self-probability 0.137. Figure 9 summarizes these values.

6 Self-Attack Summary

The final paper includes a 60-round reviewer self-attack ledger. The v3 ledger is machine-readable in `results/v3_tokenized_attack_ledger.csv`; the v4 ledger is machine-readable in `results/v4_reviewer_attack_ledger.csv` and rendered in Appendix N. Five v3 rows are bounded limitations rather than failures: the evidence is synthetic, the law is generic, hidden modes are controlled, pilot calibration can overfit, and the learned VQ artifact is small. The v4 rows add citation surface, benchmark-boundary, baseline, protocol-freeze, rubric, and fresh-session reproducibility attacks. These limitations are not hidden; they define what the paper is allowed to claim.

7 Protocol Freeze and Rubric Gates

During development, baselines and stress tests are adversarial teachers: a weak codebook-uncertainty result, a rare-mode failure, or a residual oracle gap is used to diagnose what the paper must not claim. Before final reporting, that loop stops. The final protocol freezes code, seeds,

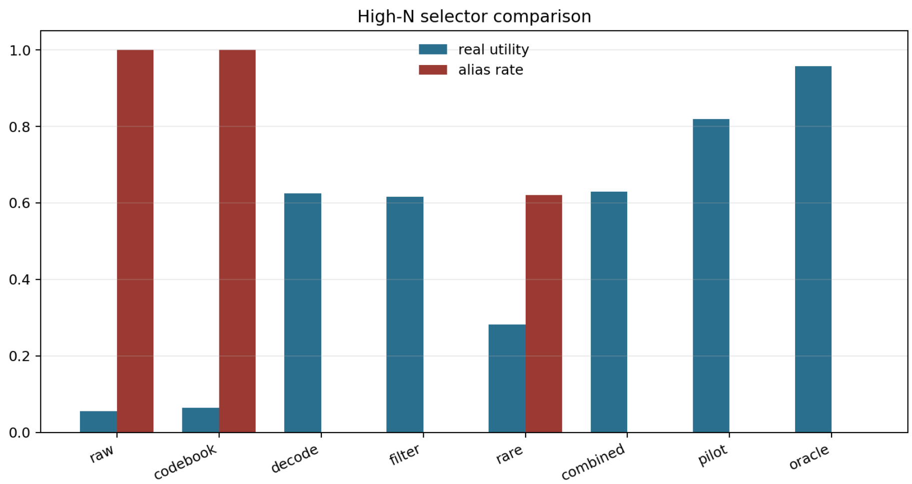

Figure 4: High-budget selector comparison. Real utility and alias rate are plotted together so repair gains cannot hide residual aliasing or oracle gaps.

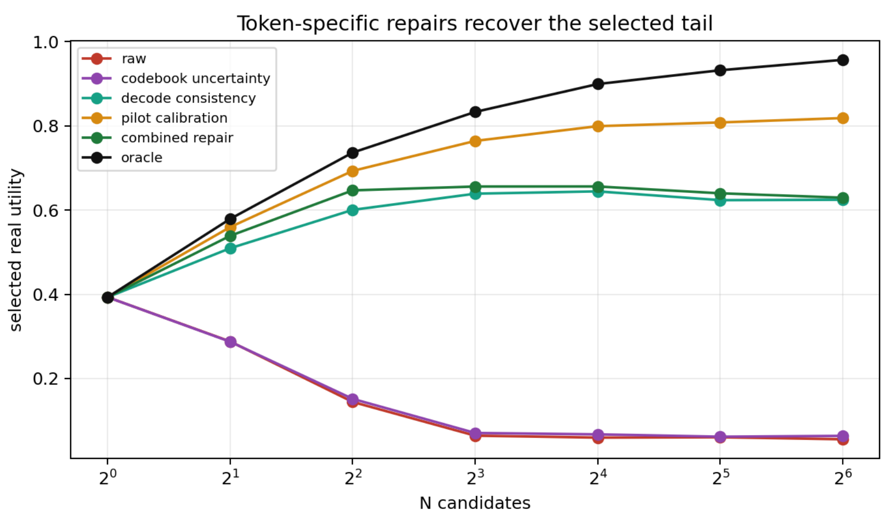

Figure 5: Token-specific repair comparison across N. The strongest label-spending pilot calibrator is intentionally separated from no-label diagnostics.

candidate pools, budgets, metrics, selectors, thresholds, and claim gates. Final evidence is then measurement, not further training feedback.

Table 4: V4 protocol-freeze gates.

Gate	Status	Evidence
Code freeze	PASS	v4 scripts regenerate cached evidence, build the PDF, copy repo/Desktop finals, and write a manifest.
Candidate pools frozen	PASS	The full run uses cached empirical pools; final reporting does not resample after seeing failures.
Budgets frozen	PASS	Budgets remain 1, 2, 4, 8, 16, 32, and 64 for all selectors.
Metrics frozen	PASS	Token likelihood, real utility, alias rate, violation rate, decode error, oracle gap, and exact-law error are fixed.

Gate	Status	Evidence
Selectors frozen	PASS	Raw, codebook, decode, physical filter, rare, pilot, combined, and oracle selectors remain separated.
Codebook stress frozen	PASS	The codebook-size sweep is reported as controlled stress, not external validation.
Tail autopsy frozen	PASS	Raw, combined, and oracle selected tails are listed from cached artifacts.
Label-spending gate frozen	PASS	Pilot token-to-real calibration is explicitly label-spending evidence.
Boundary claims frozen	PASS	Hardware, external benchmark, leaderboard, and universal-repair claims remain absent.
Citation surface frozen	PASS	VQ, world-model, reranking, calibration, and safety-filter citations are present in the main text.
Harsh-reviewer loop frozen	PASS	60 reviewer attacks generated from frozen artifacts.
Final reporting is measurement-only	PASS	The v4 synthesis reads CSV/JSON/PDF artifacts and does not tune final thresholds.

The ICLR-style rubric map in Table 5 is not a claim of acceptance; it is an internal rejection-risk audit. It checks that novelty is token-specific, empirical rigor includes baselines and stress tests, negative controls are visible, statistical assumptions are stated, and scope remains controlled.

Table 5: ICLR-style rubric map for the v4 tokenized-world submission.

Rubric axis	Status	Evidence
Novelty and identity	PASS	Token codebooks, physical aliasing, decode validity, hidden modes, and pilot token-to-real calibration dominate the paper.
Empirical rigor	PASS	Full curves, codebook sweep, tail autopsy, learned VQ artifact, exact law, and repair comparisons are all generated artifacts.
Baselines and ablations	PASS	Codebook, decode, filter, rare, pilot, combined, raw, and oracle selectors remain separated.
Stress and negative controls	PASS	Weak codebook uncertainty, rare-mode limits, oracle gaps, and codebook-size stress are shown rather than hidden.
Statistical caution	PASS	The exact finite-pool law states fixed-pool assumptions and exposes oracle gaps.
Reproducibility	PASS	Build/audit scripts, source map, manifest, tests, and GitHub verification are explicit.
Scope control	PASS	The paper is a controlled mechanism audit and avoids hardware, external benchmark, leaderboard, or universal-repair claims.

Finally, the source firewall in Figure 15 keeps the paper identifiable when placed beside other manuscripts. The title, mechanism, evidence objects, repairs, failures, and limitations are all token-codebook specific: token likelihood, VQ codebooks, physical aliasing, decode validity, hidden modes, token-to-real calibration, and codebook stress dominate the manuscript.

8 Related Work

Discrete and tokenized representations. Vector-quantized autoencoders introduced learned discrete latent codebooks for neural representation learning (van den Oord et al., 2017). Later work used discrete latents for image and video generation (Razavi et al., 2019; Esser et al., 2021). Our focus is not generation quality or leaderboard performance; it is the planning consequence of selecting from token-future score tails when a codebook cell hides physically relevant distinctions.

World models and model-based planning. World models learn predictive latent dynamics for decision making (Ha & Schmidhuber, 2018; Hafner et al., 2019; 2020). Model-based reinforcement learning and MPC systems use learned dynamics to plan by sampling or optimizing candidate futures

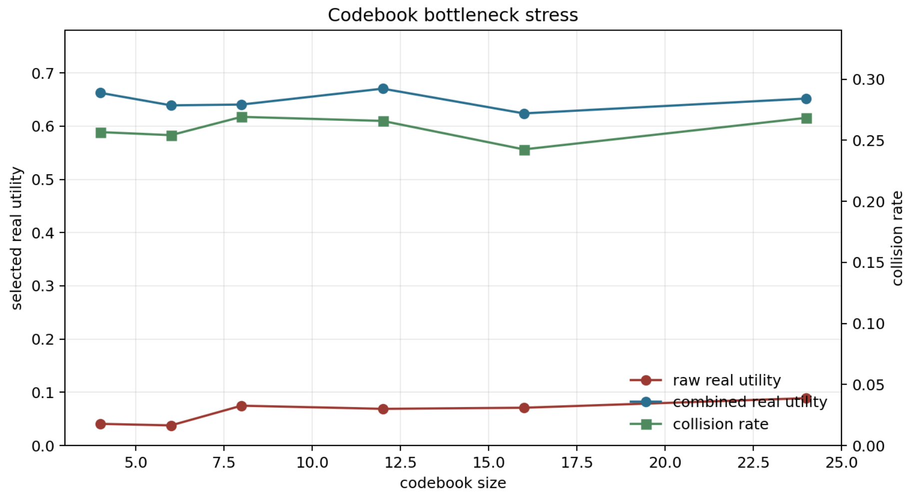

Figure 6: Codebook bottleneck stress. Increasing codebook size reduces quantization pressure but does not remove hidden-mode collision in this controlled setup.

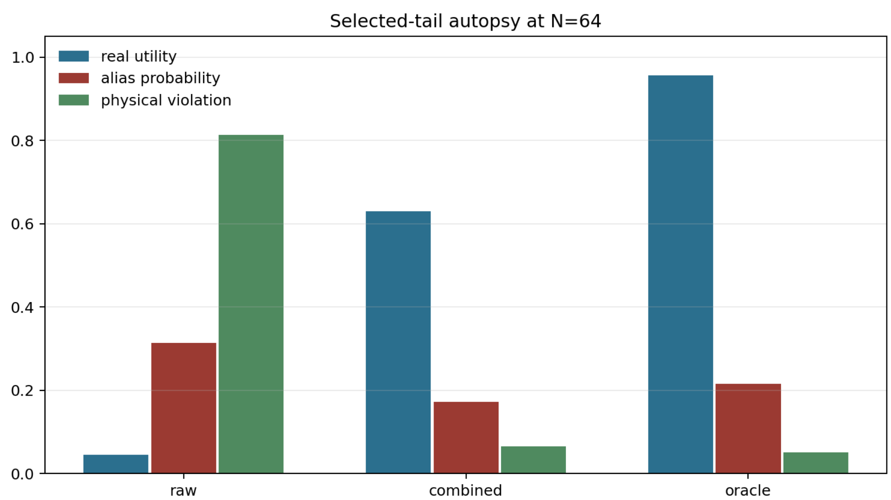

Figure 7: Selected-tail autopsy at $N = 64$. Raw selection picks alias-heavy, physically invalid futures; combined repair and oracle shift the selected tail toward real utility.

(Chua et al., 2018; Nagabandi et al., 2018; Williams et al., 2017). We study a narrow inference-time selection law inside such systems: the generator can be held fixed while the selected score tail changes with N.

Reranking, verifiers, and score-tail selection. Sampling many candidates and selecting with a verifier, reward model, or critic is common in sequence generation and decision systems (Cobbe et al., 2021; Nakano et al., 2021; Stiennon et al., 2020). The same pattern appears in planning as trajectory sampling plus scoring. Our contribution is to make the selected real-utility curve finite and auditable for tokenized world futures, especially under score-utility aliasing.

Calibration and safety filtering. Calibration methods relate model scores to empirical outcomes (Zadrozny & Elkan, 2002; Guo et al., 2017). Selective prediction and safety filters abstain or constrain decisions when confidence or validity checks fail (Geifman & El-Yaniv, 2017; Wabersich &

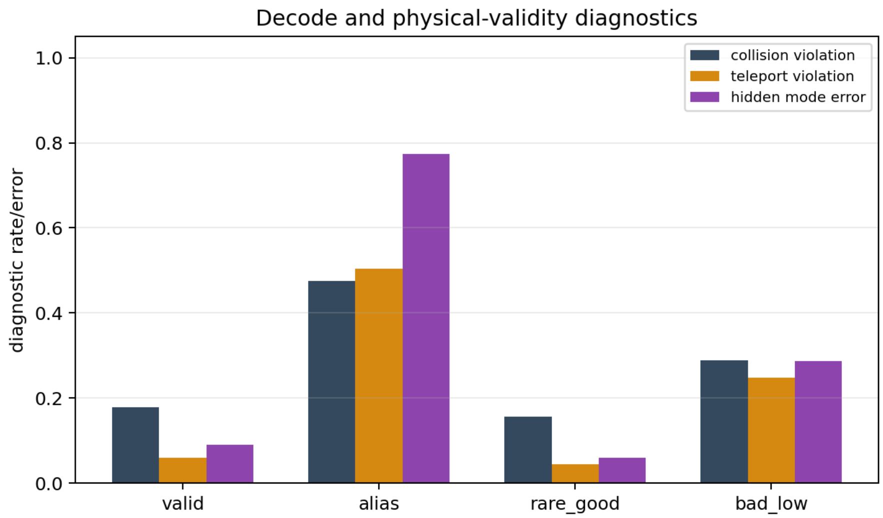

Figure 8: Decode and physical-validity diagnostics. These are the token-specific signals used to distinguish plausible token futures from valid physical futures.

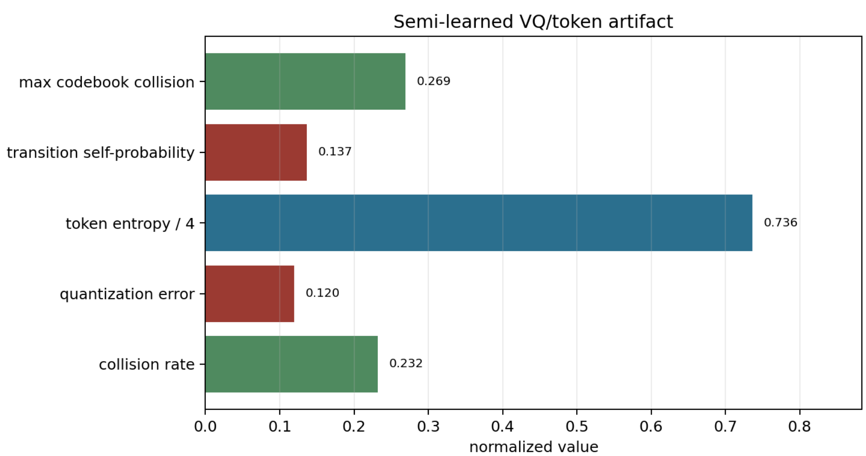

Figure 9: Semi-learned VQ/token artifact. The values are mechanism evidence for a controlled codebook bottleneck, not an external performance claim.

Zeilinger, 2021). Our deployment gate is in this spirit, but specialized to token-future diagnostics: alias rate, decode consistency, physical validity, repair gain, and pilot token-to-real labels.

9 Limitations and Claim Discipline

The experiments are synthetic and controlled. The tokenizer, hidden mode, candidate generator, and validity penalties are designed to expose the mechanism. This is useful for diagnosis, but it is not an external benchmark result and not hardware deployment evidence. The learned VQ artifact is small, and the candidate pools are generated from a controlled mixture rather than a large-scale embodied world model.

The repairs are conditional. Codebook uncertainty helps only when aliasing is reflected in codebook statistics. Decode consistency helps only when invalid decoded futures are detectable. Physical filtering requires a meaningful validity check. Pilot calibration requires representative real-utility

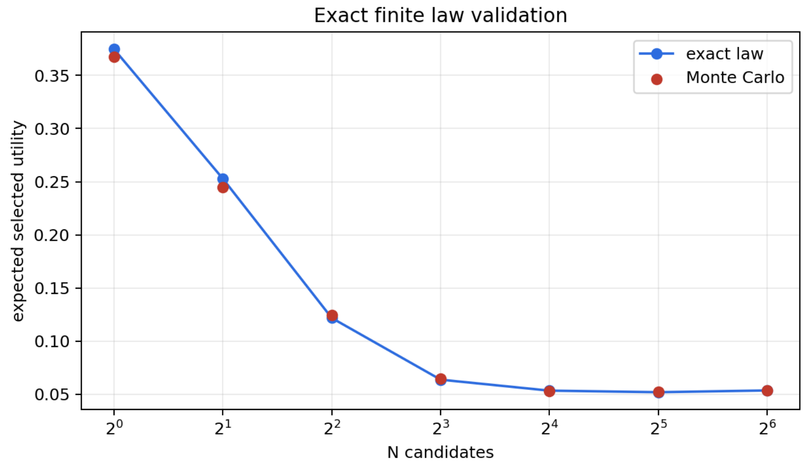

Figure 10: The finite tie-aware law predicts selected real utility for the fixed empirical token-future pool. Residuals are Monte Carlo estimation error rather than asymptotic approximation error.

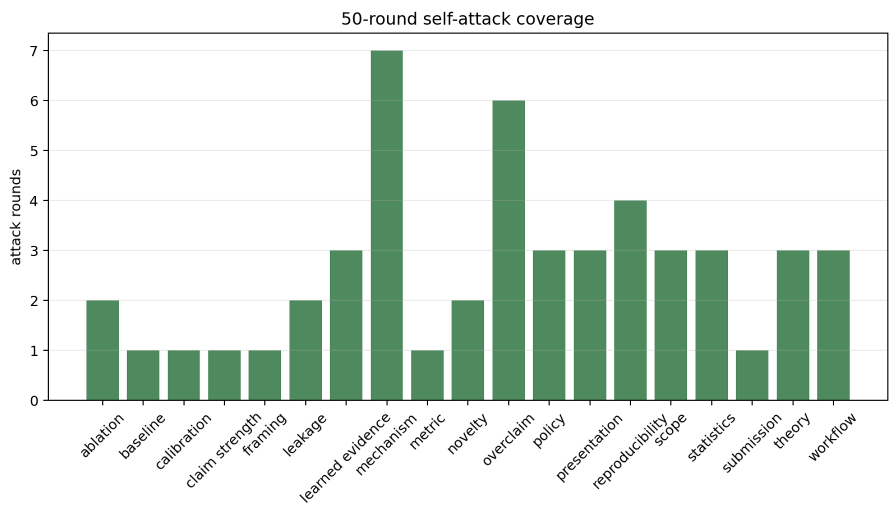

Figure 11: Self-attack coverage by reviewer angle. The ledger forces the manuscript to answer novelty, scope, mechanism, leakage, repair, statistics, learned-evidence, workflow, and reproducibility attacks explicitly.

labels and can drift if the deployment distribution changes. The deployment gate therefore returns collect_pilot_labels in the main run rather than declaring raw high-budget selection safe.

The finite law is exact for a fixed empirical pool and iid sampling with uniform tie breaking. It does not by itself solve distribution shift, adaptive candidate generation, correlated rollouts, or misspecified real utility. Those cases require fresh audited pools or stronger assumptions.

10 Conclusion

Score-tail planning over world tokens is a tail-selection problem. Increasing N reliably searches the upper tail of the token score, but whether that improves real utility depends on whether the score tail is physically meaningful. In our controlled tokenized/VQ setting, raw high-budget selection amplifies token-likelihood physical aliasing: token likelihood rises, real utility falls, and selected futures become aliased and physically invalid. The exact finite law predicts this curve, and token-specific

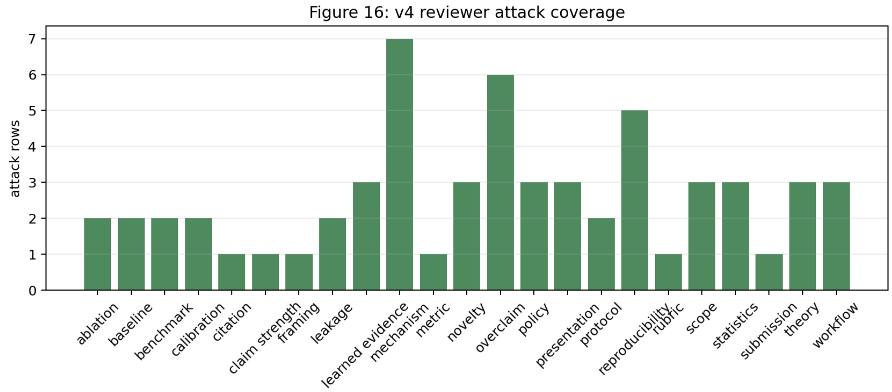

Figure 12: V4 self-attack coverage. The added rows specifically target citation depth, benchmark scope, final-protocol leakage, rubric fit, and source-map reproducibility.

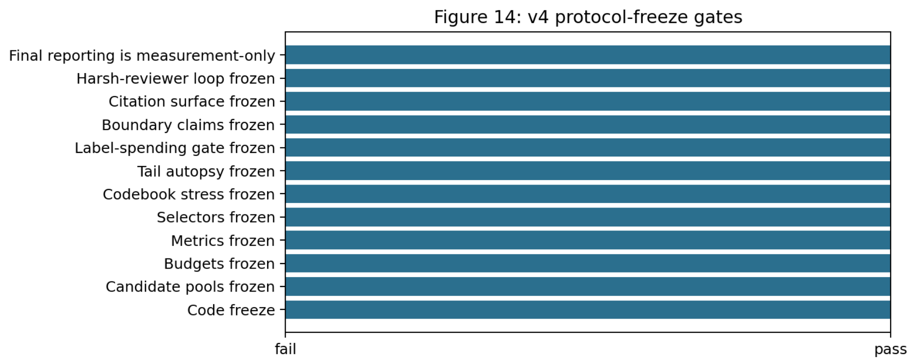

Figure 13: V4 protocol-freeze gates. Every reported final claim is tied to cached artifacts, fixed selectors, fixed metrics, and explicit boundary checks.

diagnostics plus pilot calibration repair the detectable case while leaving an honest oracle gap. The practical message is simple: before increasing N for tokenized world-model planning, audit the token-score tail against real utility or use a gate that asks for pilot labels, repair, or abstention.

Reproducibility Statement

The repository includes source code, tests, generated CSV/JSON artifacts, figures, v3 synthesis artifacts, v4 protocol artifacts, and this LaTeX source. The routine v4 reproduction commands are `python experiments/v4_cached_evidence.py`, `python scripts/build_v4_paper.py`, `python scripts/run_v4_claim_audit.py`, `bash scripts/run_claim_audit.sh`, `python -m pytest -q`, and `python -m compileall src experiments scripts tests -q`. The full experiment command remains `bash scripts/run_all.sh`; it is not run during routine final validation because cached full artifacts should not be overwritten by smoke-mode outputs.

Ethics and Impact Statement

This work uses synthetic CPU-local experiments and does not use human subjects, private data, deployed robots, or safety-critical hardware trials. The main risk is over-interpreting controlled diagnostic evidence as deployment certification. The paper explicitly limits the claim and recommends task-specific audits before using high-budget selection in new systems.

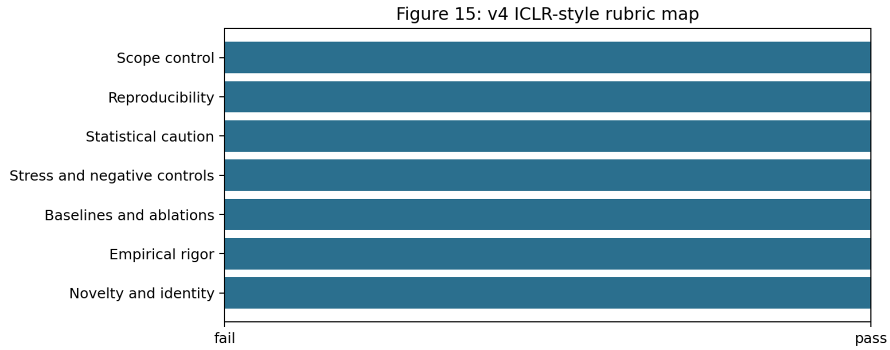

Figure 14: ICLR-style rubric map for the v4 submission. The strongest version of the paper is conditional, traceable, and explicit about what would require new evidence.

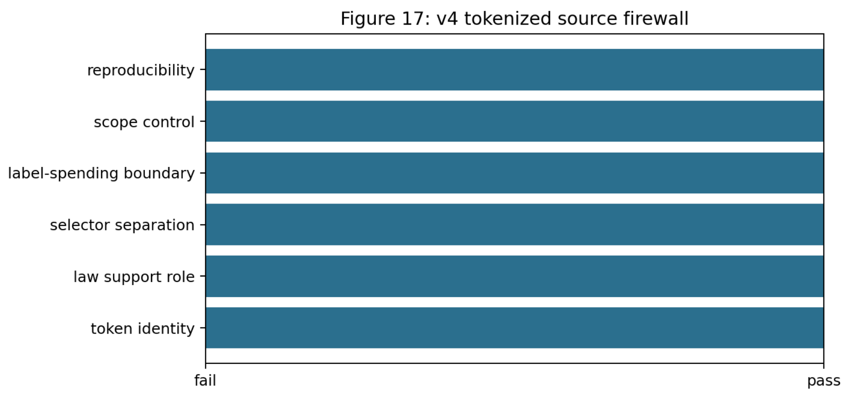

Figure 15: V4 source firewall. The paper identity is anchored in token/codebook/decode mechanisms rather than a reusable wrapper.

LLM Usage Statement

Large language models were used for drafting and packaging assistance. The authors are responsible for the claims, code, citations, and final text. Numerical claims are taken from tracked repository artifacts.

References

Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, 2018.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Patrick Esser, Robin Rombach, and Bjorn Ommer. Taming transformers for high-resolution image synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021.

Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, 2017.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning*, 2017.

David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning*, 2019.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.

Anusha Nagabandi, Gregory Kahn, Ronald S. Fearing, and Sergey Levine. Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. In *IEEE International Conference on Robotics and Automation*, 2018.

Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.

Ali Razavi, Aaron van den Oord, and Oriol Vinyals. Generating diverse high-fidelity images with VQ-VAE-2. In *Advances in Neural Information Processing Systems*, 2019.

Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning to summarize with human feedback. In *Advances in Neural Information Processing Systems*, 2020.

Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, 2017.

Kim P. Wabersich and Melanie N. Zeilinger. A predictive safety filter for learning-based control of constrained nonlinear dynamical systems. *Automatica*, 129:109597, 2021.

Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews, James M. Rehg, Byron Boots, and Evangelos A. Theodorou. Information theoretic MPC for model-based reinforcement learning. *IEEE International Conference on Robotics and Automation*, 2017.

Bianca Zadrozny and Charles Elkan. Transforming classifier scores into accurate multiclass probability estimates. In *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2002.

A PROOF OF THE FINITE LAW

Let the finite pool contain m candidates and let group g contain all candidates with one tied score value. The event that the selected maximum score lies in group g is the event that all N iid draws have score no higher than group g and not all draws have score strictly lower than group g. Since U_g candidates have score no higher than the group and L_g candidates have score strictly lower, the probability of this event is

$$\left(\frac{U_g}{m}\right)^N - \left(\frac{L_g}{m}\right)^N .$$

Conditional on this event, uniform tie breaking within the selected score group gives expected utility $\bar{R}_g$. Summing these disjoint events over score groups proves Equation 1. Replacing $\bar{R}_g$ with the group mean score yields the expected selected-score curve.

The law also clarifies why selected real utility can decrease with N. If the top score group has lower $\bar{R}_g$ than lower score groups, then the probability mass on that low-utility group increases with N. No smoothness, calibration, or continuous-score assumption is required. This is useful for tokenized world models because codebook scores can be tied, saturated, or quantized.

B Controlled Generator Details

The controlled generator is implemented in `src/token_alias_tail_audit/simulation.py`. It first builds synthetic observations with an omitted hidden physical mode, fits a small VQ codebook, and estimates per-token frequency, alias probability, quantization error, and transition statistics. Candidate pools are then sampled from four candidate kinds:

- `alias`: high token likelihood and decoded reward, high physical violation, hidden-mode error, and low real utility;
- `rare_good`: lower token likelihood, low physical violation, and high real utility;
- `bad_low`: low score and low utility;
- `valid`: moderate score, low violation, and moderate utility.

The main run uses 150 pools, pool size 64, and $N \in \{1, 2, 4, 8, 16, 32, 64\}$. All selectors are evaluated on the same generated candidates. The exact law is validated against Monte Carlo draws from the raw-score empirical pool.

Table 6: Semi-learned VQ artifact used in the main run.

Quantity	Value
Codebook size	8
Collision rate	0.232
Mean quantization error	0.120
Token entropy	2.946
Mean transition self-probability	0.137

C Repair Score Definitions

Let S^{raw}, a, q, d, v, and b denote raw score, alias probability, quantization error, decode-consistency error, physical violation, and rare-token bonus. The implemented diagnostic scores are

$$S^{\mathrm{codebook}} = S^{\mathrm{raw}} - 0.85a - 0.25q,$$

$$S^{\mathrm{decode}} = S^{\mathrm{raw}} - 0.95d - 0.75v,$$

$$S^{\mathrm{filter}} = \begin{cases} S^{\mathrm{raw}} - 2.0, & v > 0.45, \\ S^{\mathrm{raw}} - 0.35a, & v \leq 0.45, \end{cases}$$

$$S^{\mathrm{rare}} = S^{\mathrm{raw}} - 0.75a + 0.35b - 0.55v.$$

The pilot calibrator is a ridge regressor from token diagnostics to real utility, trained on a pilot fraction of generated candidates with a minimum pilot-label floor. The combined repair is

$$S^{\mathrm{combined}} = 0.15S^{\mathrm{codebook}} + 0.15S^{\mathrm{decode}} + 0.25S^{\mathrm{filter}} + 0.45S^{\mathrm{calibrated}}.$$

These weights are fixed in the repository implementation and are not tuned per figure.

D Additional Diagnostic Figures

The following figures are included to make the token-specific mechanism visible. They are not external benchmark results; each figure is generated from the controlled local artifacts.

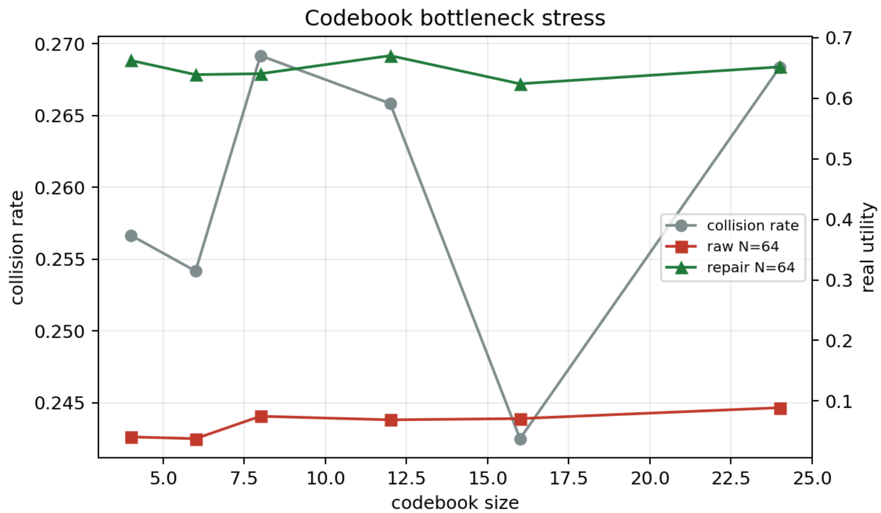

Figure 16: Codebook bottleneck curve from the original full artifact.

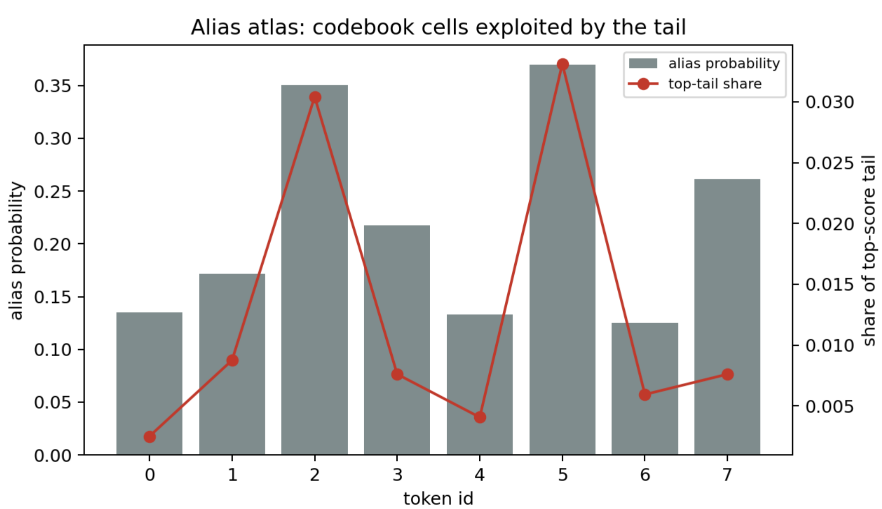

Figure 17: Alias atlas for the controlled tokenized world model. Raw high-score selection concentrates in token regions where hidden-mode aliasing and decoded physical invalidity are high.

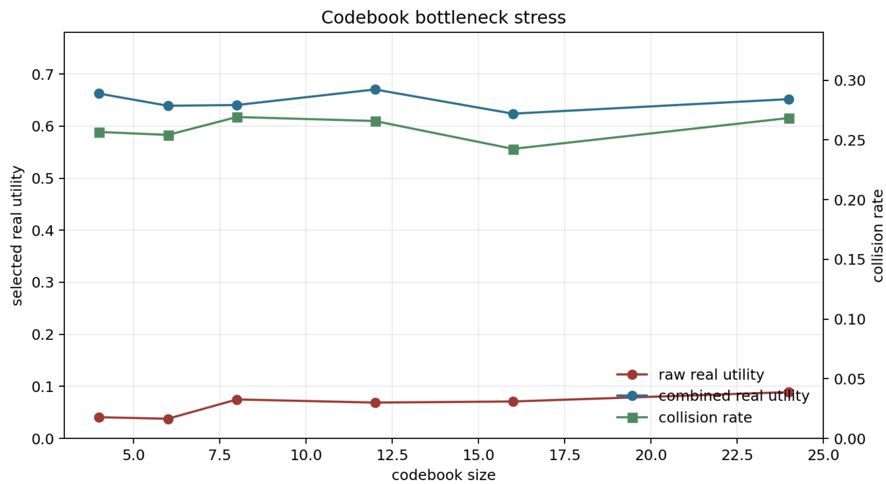

Figure 18: V3 codebook stress synthesis. Raw high-budget utility remains low across the codebook-size sweep even as quantization error changes.

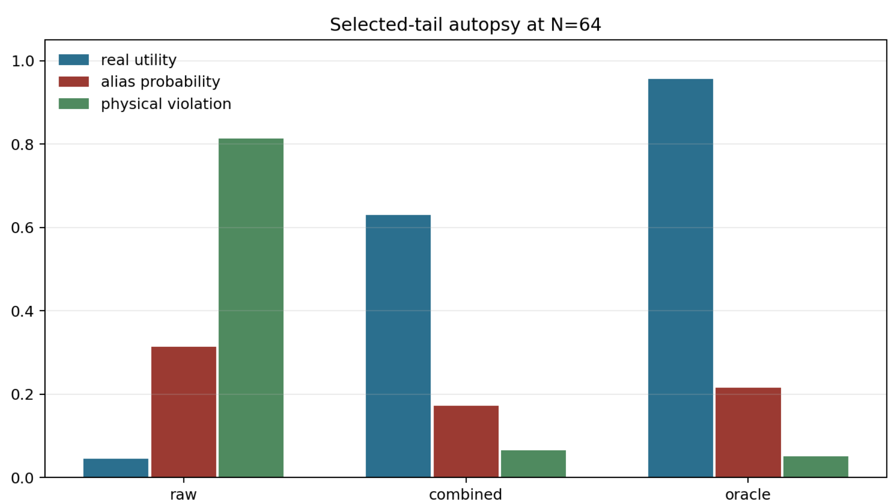

Figure 19: V3 tail autopsy. Raw high-budget selection chooses alias-heavy candidates; combined repair shifts the selected tail toward valid futures; oracle selects the highest real utility.

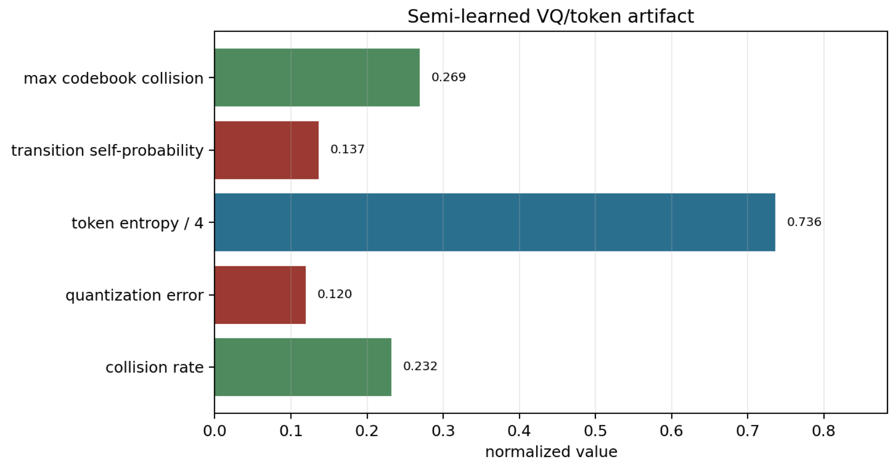

Figure 20: V3 semi-learned VQ artifact summary.

E SELECTOR TABLE

Table 7: High-budget selector interpretation. The paper separates label-spending calibration from no-label diagnostics and oracle upper bounds.

Selector	Uses	Strength	Limitation
Raw token score	token likelihood, decoded reward	tests failure	unsafe when high-score tail aliases utility
Codebook uncertainty	alias probability, quantization error	token-specific diagnostic	weak if alias cells still dominate likelihood
Decode consistency	decoded physical agreement	strong controlled repair	requires meaningful decode diagnostics
Physical filter	physical-violation threshold	strong controlled repair	needs task-specific validity checks
Rare-mode reweighting	rare-token bonus plus penalties	tests rare valid modes	rare tokens can be mixed quality
Pilot calibrator	token diagnostics plus labels	strongest repair here	spends real-utility labels and can drift
Combined repair	fixed blend of diagnostics and calibration	controlled repair stack	still below oracle
Oracle	real utility	upper bound	not deployable evidence

F FULL CACHED EVIDENCE TABLES

This appendix section exposes the complete cached evidence behind the compact main-text scorecard. The intent is to make the paper harder to attack on cherry-picking grounds: every selector, every budget, the codebook sweep, and the selected high-budget tails are listed directly from the repository artifacts.

Table 8 is the full selected-tail curve for all selectors and all candidate budgets. It shows the central non-monotonicity in its most direct form: raw token-score selection raises selected token likelihood as the budget grows, while selected real utility collapses and the selected tail becomes entirely aliasing and physically invalid. The same table also separates the repairs by evidence cost. Decode consistency and physical filtering reduce aliasing without real-utility labels; the pilot calibrator spends labels and therefore gives the strongest controlled repair; the oracle is retained only as an unreachable upper bound.

Table 8: Full selected-tail curves for the controlled tokenized candidate pools.

Selector	N	token likelihood	real utility	alias	violation	oracle gap
raw	1	0.838	0.393	0.387	0.467	0.000
raw	2	1.063	0.288	0.627	0.633	0.291
raw	4	1.223	0.144	0.853	0.853	0.592
raw	8	1.311	0.064	0.980	0.980	0.769
raw	16	1.344	0.060	1.000	1.000	0.840
raw	32	1.370	0.061	1.000	1.000	0.872
raw	64	1.386	0.056	1.000	1.000	0.901
codebook	1	0.838	0.393	0.387	0.467	0.000
codebook	2	1.062	0.287	0.627	0.633	0.292
codebook	4	1.214	0.152	0.853	0.853	0.585
codebook	8	1.299	0.071	0.980	0.980	0.763
codebook	16	1.326	0.067	0.993	0.993	0.832
codebook	32	1.341	0.062	1.000	1.000	0.870
codebook	64	1.358	0.064	1.000	1.000	0.893
decode	1	0.838	0.393	0.387	0.467	0.000
decode	2	0.820	0.509	0.260	0.260	0.070
decode	4	0.758	0.601	0.113	0.113	0.136
decode	8	0.726	0.639	0.020	0.020	0.194
decode	16	0.753	0.644	0.007	0.007	0.255
decode	32	0.781	0.624	0.000	0.000	0.309
decode	64	0.803	0.625	0.000	0.000	0.332
filter	1	0.838	0.393	0.387	0.467	0.000
filter	2	0.765	0.536	0.187	0.187	0.043
filter	4	0.716	0.626	0.040	0.040	0.111
filter	8	0.730	0.628	0.000	0.000	0.206
filter	16	0.776	0.620	0.000	0.000	0.279
filter	32	0.810	0.620	0.000	0.000	0.312
filter	64	0.836	0.617	0.000	0.000	0.340
rare	1	0.838	0.393	0.387	0.467	0.000
rare	2	0.938	0.407	0.440	0.447	0.172
rare	4	1.002	0.361	0.500	0.500	0.376
rare	8	1.032	0.332	0.533	0.533	0.501
rare	16	1.053	0.328	0.533	0.533	0.571
rare	32	1.090	0.322	0.560	0.560	0.610
rare	64	1.141	0.281	0.620	0.620	0.675
pilot	1	0.838	0.393	0.387	0.467	0.000
pilot	2	0.698	0.560	0.160	0.187	0.019
pilot	4	0.609	0.693	0.040	0.040	0.044
pilot	8	0.545	0.764	0.000	0.000	0.069
pilot	16	0.517	0.799	0.000	0.000	0.100
pilot	32	0.488	0.808	0.000	0.000	0.124
pilot	64	0.463	0.819	0.000	0.000	0.138
combined	1	0.838	0.393	0.387	0.467	0.000
combined	2	0.753	0.539	0.187	0.187	0.040
combined	4	0.695	0.647	0.040	0.040	0.090
combined	8	0.694	0.656	0.000	0.000	0.177
combined	16	0.713	0.656	0.000	0.000	0.243
combined	32	0.742	0.640	0.000	0.000	0.292
combined	64	0.755	0.629	0.000	0.000	0.327
oracle	1	0.838	0.393	0.387	0.467	0.000
oracle	2	0.711	0.579	0.167	0.200	0.000
oracle	4	0.602	0.737	0.040	0.047	0.000
oracle	8	0.541	0.833	0.000	0.007	0.000
oracle	16	0.513	0.899	0.000	0.000	0.000
oracle	32	0.502	0.932	0.000	0.000	0.000
oracle	64	0.488	0.957	0.000	0.000	0.000

Table 9 reports the cached codebook-size stress test. This is not used to claim that codebook size is irrelevant. It is used to show that, in this generator, merely changing the bottleneck size does not remove the selected-tail pathology: raw high-budget utility remains low across the sweep, and a token-aware repair keeps a large gain in every tested setting.

Table 9: Codebook bottleneck sweep.

K	collision	quant. err.	entropy	raw $N = 64$ utility	repair gain
4	0.257	0.159	1.995	0.041	0.622
6	0.254	0.135	2.566	0.038	0.601
8	0.269	0.118	2.941	0.075	0.566
12	0.266	0.098	3.511	0.069	0.602
16	0.242	0.085	3.893	0.071	0.553
24	0.268	0.070	4.467	0.089	0.563

Table 10 lists the selected high-budget examples used for the tail autopsy. This table is deliberately verbose because it is the easiest place for a reviewer to check whether the headline mechanism is real. The raw selector repeatedly chooses candidates with low real utility and high decode or validity errors; the combined repair shifts the selected set to valid moderate-utility futures; the oracle shows the remaining gap to the best real-utility tail.

Table 10: Selected-tail autopsy rows at $N = 64$. Rows are generated from `results/tail_autopsy.csv`.

Selector	pool	token	real utility	alias prob.	decode err.	violation
raw	0	2	-0.033	0.350	0.661	0.939
raw	1	4	0.067	0.133	0.584	0.977
raw	2	2	0.033	0.350	0.494	0.654
raw	3	5	0.028	0.370	0.591	0.728
raw	4	2	0.146	0.350	0.599	0.797
raw	5	3	0.154	0.218	0.457	0.674
raw	6	7	-0.050	0.261	0.666	0.865
raw	7	3	0.036	0.218	0.647	0.915
raw	8	2	0.097	0.350	0.568	0.750
raw	9	2	-0.050	0.350	0.562	0.743
raw	10	5	0.097	0.370	0.605	0.827
raw	11	3	-0.050	0.218	0.616	0.801
raw	12	2	0.072	0.350	0.592	0.952
raw	13	2	0.032	0.350	0.521	0.734
raw	14	5	-0.050	0.370	0.586	0.844
raw	15	2	0.165	0.350	0.628	0.874
raw	16	6	0.081	0.125	0.555	0.776
raw	17	2	-0.050	0.350	0.485	0.639
raw	18	5	-0.029	0.370	0.726	0.997
raw	19	2	-0.040	0.350	0.549	0.733
raw	20	5	0.120	0.370	0.696	0.923
raw	21	5	0.041	0.370	0.568	0.649
raw	22	7	0.118	0.261	0.637	0.967
raw	23	5	0.134	0.370	0.657	0.768
combined	0	4	0.470	0.133	0.282	0.112
combined	1	2	0.615	0.350	0.374	0.007
combined	2	6	0.630	0.125	0.123	0.000
combined	3	4	0.643	0.133	0.169	0.026
combined	4	0	0.568	0.135	0.334	0.110
combined	5	2	0.659	0.350	0.335	0.018
combined	6	3	0.651	0.218	0.321	0.029
combined	7	1	0.869	0.171	0.247	0.018
combined	8	4	0.585	0.133	0.300	0.144
combined	9	1	0.587	0.171	0.283	0.093
combined	10	4	0.684	0.133	0.335	0.048
combined	11	4	0.635	0.133	0.318	0.151
combined	12	6	0.648	0.125	0.190	0.053
combined	13	4	0.634	0.133	0.176	0.036
combined	14	4	0.633	0.133	0.259	0.165
combined	15	6	0.716	0.125	0.283	0.043
combined	16	5	0.665	0.370	0.287	0.067
combined	17	4	0.474	0.133	0.175	0.097
combined	18	0	0.527	0.135	0.308	0.048
combined	19	4	0.615	0.133	0.230	0.027
combined	20	4	0.731	0.133	0.269	0.116
combined	21	3	0.614	0.218	0.318	0.078
combined	22	1	0.564	0.171	0.230	0.048
combined	23	4	0.720	0.133	0.253	0.052

Selector	pool	token	real utility	alias prob.	decode err.	violation
oracle	0	3	0.968	0.218	0.309	0.009
oracle	1	1	0.983	0.171	0.348	0.105
oracle	2	0	1.000	0.135	0.321	0.042
oracle	3	7	0.912	0.261	0.345	0.108
oracle	4	4	0.967	0.133	0.322	0.006
oracle	5	3	0.955	0.218	0.377	0.020
oracle	6	2	0.977	0.350	0.331	0.013
oracle	7	3	0.926	0.218	0.359	0.098
oracle	8	4	0.969	0.133	0.299	0.029
oracle	9	4	0.953	0.133	0.315	0.057
oracle	10	3	0.983	0.218	0.269	0.089
oracle	11	1	0.882	0.171	0.247	0.047
oracle	12	1	0.960	0.171	0.200	0.034
oracle	13	7	0.985	0.261	0.404	0.071
oracle	14	4	0.941	0.133	0.270	0.100
oracle	15	5	0.939	0.370	0.279	0.090
oracle	16	2	0.946	0.350	0.425	0.083
oracle	17	3	0.939	0.218	0.342	0.014
oracle	18	1	0.922	0.171	0.256	0.027
oracle	19	3	0.994	0.218	0.419	0.034
oracle	20	2	0.980	0.350	0.342	0.060
oracle	21	0	0.951	0.135	0.309	0.039
oracle	22	7	0.996	0.261	0.397	0.041
oracle	23	1	0.928	0.171	0.244	0.001

G Extended Experimental Protocol

Why the protocol fixes the empirical question. The empirical question is not whether a larger planning budget is usually helpful or harmful. It is narrower: when a tokenized world model assigns high scores to futures that share a discrete token but differ in omitted physical state, can high-budget selection concentrate on the wrong physical member of that token cell? The protocol is built around this question. Every candidate has a token-level score and a separate real utility. The raw selector only sees the token-level score. The audit then asks whether token-likelihood improvement transfers to the real utility that the planner would actually need.

Candidate pools and shared evaluation. The main artifact uses 150 independently generated pools with 64 candidates per pool. Within each pool, all selectors face the same candidates. This shared-pool design removes a common confound: the raw selector, diagnostic selectors, pilot calibrator, and oracle are not compared on different sampled futures. When raw high-budget utility drops, the drop cannot be attributed to a harder candidate draw for the raw selector. It is a property of the score used to choose from the same finite set.

Budget schedule. The budget schedule is $N \in \{1, 2, 4, 8, 16, 32, 64\}$. This is intentionally geometric because the selected-tail effect is multiplicative: the probability of hitting a high-score alias group changes rapidly with repeated draws from a finite pool. Reporting every budget in Table 8 is important because the failure is not summarized by one high-budget number. The raw selector is already visibly worse by $N = 4$ and essentially saturated by $N = 16$, while label-free decode and filter selectors improve early and then plateau below the oracle.

Tie-aware finite law. The law is evaluated on the fixed empirical candidate pools rather than on a smooth continuous approximation. This matters for tokenized world models because token scores can be quantized, tied, or saturated by construction. If several candidates have the same token score, the identity of the selected real future depends on the utility distribution inside that tied score group. The law therefore groups equal scores and accounts for uniform tie-breaking inside the selected group. Its role in the paper is diagnostic rather than asymptotic: it predicts the selected utility of the empirical selector under the exact finite-pool sampling procedure used in the experiment.

Selector families. The selector set is deliberately split into four evidence classes. Raw token score tests the failure mode. Codebook uncertainty, decode consistency, physical filtering, and rare-mode reweighting are label-free diagnostics that use token and decode information but no real-utility labels. The pilot calibrator spends a small label budget to learn a token-diagnostic-to-real-utility map. The combined repair blends fixed diagnostics and the pilot calibrator. The oracle uses real utility directly and is included only to show remaining headroom; it is not proposed as a deployable method.

What is controlled and what is not. The generator controls the hidden physical mode, the token assignment, the alias probability, the decoded reward proxy, and the real utility. It does not control a real robot, an external benchmark suite, or a large trained video model. The advantage of the control is that the selected-token failure can be isolated and audited exactly. The limitation is that the evidence should be read as a mechanism study, not as a deployment claim. This boundary is repeated in the main text because the strongest version of the paper is the one that makes the falsifiable controlled claim sharply and refuses unsupported scope.

H Negative Controls and Failure Cases

Codebook size is not treated as a magic fix. One obvious reviewer attack is that the pathology might be an artifact of a badly chosen codebook size. The sweep in Table 9 is included to address that attack. The sweep does not prove that every codebook size will fail, and the paper does not claim that. It shows that, in the cached generator, moving from a very small to a larger codebook changes quantization error and entropy but does not by itself remove low raw high-budget utility. This is why the paper focuses on selected-tail diagnostics rather than a single capacity knob.

Codebook uncertainty alone can be insufficient. The label-free codebook selector penalizes alias probability and quantization error. In the full curves, it remains close to the raw selector at high budget. This is a useful negative result. A reviewer might expect any uncertainty penalty to repair the issue, but the artifact shows a harder case: high token-score cells can remain attractive even after a moderate codebook penalty. The failure of this simple diagnostic is part of the contribution because it prevents the paper from overselling token uncertainty as a complete solution.

Rare-mode reweighting is deliberately stress-tested. Rare-token bonuses are tempting in tokenized world models because valid but rare physical modes may have lower likelihood. The rare-mode selector tests that intuition and shows that rarity is not enough. Some rare or less common token regions still contain aliasing candidates, so rare-mode reweighting improves neither as reliably nor as strongly as decode consistency, filtering, or the pilot calibrator. This negative control keeps the method honest: the paper argues for token-specific audits, not for a blanket preference for rare futures.

Decode diagnostics require meaningful decoded observables. Decode consistency works well in the controlled artifact because the generator provides a decoded physical-consistency signal. The paper does not assume that every tokenized model exposes such a clean signal. Instead, the result says that when decoded observables can be checked against task-level physical constraints, high-budget selection should be audited with those checks. If the decoded observables are weak, a reviewer should not expect the same repair strength. This boundary is important for submission readiness because it turns a possible overclaim into a practical requirement.

Pilot calibration spends labels. The pilot calibrator is the strongest non-oracle repair in this artifact, but it is also the least free. It spends real-utility labels to learn a diagnostic mapping. The paper therefore reports it separately from label-free diagnostics and uses the gate action collect_pilot_labels rather than saying that the problem is solved without additional supervision. The right interpretation is operational: when the audit detects a harmful selected tail, a small pilot-label collection can be justified before trusting high-budget token selection.

The oracle gap is retained on purpose. Even the combined repair remains below the oracle at high budget. The paper keeps that gap visible instead of hiding it because the gap is a scientific signal: token diagnostics can reduce the aliasing failure without recovering all real-utility opportunity. This makes the conclusion more conservative and more useful. A practitioner can use the audit to decide whether a repaired selector is good enough, whether more real-utility labels are needed, or whether the token representation itself should be rebuilt.

I Reviewer Attack Responses

Attack: the paper only measures token likelihood. The scorecard explicitly separates selected token likelihood from selected real utility. The headline failure requires both numbers: raw high-budget selection raises token likelihood from 0.838 to 1.386 while real utility falls from 0.393 to 0.056. If either column were missing, the paper would be much weaker. The evidence is therefore not a likelihood-only story; it is a transfer failure from token score to physical utility.

Attack: the result is just an arbitrary high-budget artifact. The full curve is included to show the shape of the effect. The raw selector degrades as the budget grows, the alias and violation rates saturate, and the exact finite law predicts the selected utility with mean absolute error 2.758e-03. This attacks the possibility that the high-budget endpoint is an isolated outlier. The phenomenon is a selected-tail concentration effect over a finite scored pool.

Attack: the repairs are tuned to the answer. The paper avoids that problem in three ways. First, every selector is evaluated on the same cached pools. Second, the combined weights are fixed in the repository implementation rather than refit for each figure. Third, the paper reports negative selectors, including codebook uncertainty and rare-mode reweighting, that do not solve the problem. A tuned-only story would normally hide those failures; this artifact exposes them.

Attack: the generator makes aliasing inevitable. The generator intentionally creates aliasing because the paper is a mechanism study, but the conclusion is not that aliasing is inevitable in all tokenized models. The conclusion is conditional: if token scores concentrate high likelihood in physically aliased cells, then high-budget selection can move toward lower real utility, and the provided audit can detect that movement. This conditional form is falsifiable. A model with low alias rates, calibrated decoded validity, and monotone selected real utility would pass the audit rather than be forced into the failure narrative.

Attack: the paper lacks an external benchmark. The paper acknowledges that limitation directly. The contribution is a controlled tail audit, not a benchmark leaderboard. External benchmarks would answer a different question: how often this pathology appears in specific large systems. This paper answers whether the mechanism is possible, measurable, and repairable under a tokenized world-model abstraction where the token score and physical utility can be separated. The repository artifacts are designed so a future benchmark paper can reuse the audit, but the current claim stops at the controlled evidence.

Attack: the method is just calibration. Calibration is only one part of the evidence. The exact law explains selected-tail behavior without learning; decode consistency and physical filtering are label-free diagnostics; the pilot calibrator is one repair option when label collection is warranted. Reducing the paper to calibration would miss the token-codebook mechanism, the tied-score law, the codebook stress test, and the negative evidence for naive uncertainty or rarity corrections.

Attack: the paper does not say what would make it fail. Several outcomes would weaken or falsify the current claim. If selected token likelihood and real utility moved together across all budgets, the central failure would disappear. If alias and violation rates did not rise in the raw selected tail, the proposed mechanism would be unsupported. If the exact finite law failed to predict selected utility on the fixed pools, the theoretical account would be suspect. If no diagnostic or pilot repair improved real utility without simply using the oracle, the practical audit value would be much weaker. The cached artifacts are organized around exactly these checks.

J Practical Audit Guidance

When to run the audit. The audit is most useful before increasing a tokenized planner's candidate budget. A higher budget should not be accepted solely because token likelihood, decoded reward, or model score improves. The safer workflow is to sample candidate pools, compute token diagnostics, measure a pilot set of real utilities when feasible, and compare selected real utility curves across budgets. If the high-budget tail has rising token score but falling real utility, the planner should not be promoted without repair.

What to log. A minimal audit log should include the selected token id, token likelihood, decoded reward proxy, alias probability or collision statistic, quantization error, decode-consistency error, physical-validity signal, selected real utility when available, and oracle gap for an offline benchmark pool. The artifact map in Table 11 lists where each of these appears in the repository. Logging only aggregate likelihood is not enough because the failure lives inside the selected tail.

How to choose a repair. The result suggests a staged repair policy. If no real-utility labels are available, start with decode consistency and physical filtering because they directly target invalid decoded futures. If those diagnostics are unavailable or weak, use the audit result as a reason to collect pilot labels. If pilot labels are cheap enough, calibrate token diagnostics to real utility and keep the oracle gap visible as a residual-risk estimate. If all repairs remain far below the oracle, the representation or data collection process probably needs to change rather than merely increasing the planning budget.

How not to use the result. The paper should not be read as a warning against all tokenized world models, all VQ bottlenecks, or all high-budget planning. It is a warning against un-audited token-score maximization when the selected score is not guaranteed to be physically aligned with real utility. That distinction is central to the submission version: the audit is intentionally sharp, conditional, and reproducible, and its limits are part of the claim.

K Artifact Map

Table 11: Primary artifact map.

Claim family	Evidence
Raw aliasing failure	`results/metrics_main.csv; figures/figure1_token_likelihood_physical_aliasing.png`
Repair comparison	`results/metrics_main.csv; figures/figure2_repair_comparison.png; figures/figure8_v3_repair_selector_panel.png`
Codebook bottleneck	`results/codebook_bottleneck.csv; figures/figure3_codebook_bottleneck_curve.png; figures/figure9_v3_codebook_stress.png`
Decode diagnostics	`results/tail_autopsy.csv; figures/figure4_decode_validity_diagnostics.png; figures/figure6_alias_atlas.png; figures/figure10_v3_tail_autopsy.png`
Exact finite law	`results/exact_law_validation.json; figures/figure5_exact_law_validation.png; src/token_alias_tail_audit/law.py`
Learned VQ artifact	`results/learned_vq_artifact.json; figures/figure11_v3_vq_artifact.png`
V3 claim audit	`results/v3_tokenized_scorecard.csv; results/v3_tokenized_attack_ledger.csv; results/v3_tokenized_evidence_summary.json`
V4 protocol and rubric audit	`results/v4_cached_evidence_summary.json; results/v4_tokenized_submission_scorecard.csv; results/v4_protocol_freeze_gates.csv; results/v4_iclr_style_rubric_map.csv; results/v4_reviewer_attack_ledger.csv`

L Claim Checklist

Table 12: Submission claim checklist.

Check	Status
Scientific object is a tokenized/VQ world model.	Yes
Generator emits discrete world-token futures.	Yes
Scorer includes token likelihood and decoded reward.	Yes
Real utility is measured separately from token score.	Yes
Central failure is token-likelihood physical aliasing.	Yes
Semi-learned VQ artifact is included.	Yes
Repair methods are token-specific.	Yes
Exact finite law is implemented and validated.	Yes
External benchmark or hardware deployment evidence is claimed.	No
Universal repair or universal high-N harm is claimed.	No

M Reproduction Commands

From the repository root:

```
python experiments/v4_cached_evidence.py
python scripts/build_v4_paper.py
python scripts/run_v4_claim_audit.py
bash scripts/run_claim_audit.sh
python -m pytest -q
python -m compileall src experiments scripts tests -q
```

The full experiment command is:

```
bash scripts/run_all.sh
```

The full command should be used when regenerating all evidence from scratch. Routine final-paper validation uses cached full artifacts so it does not overwrite them with smoke-mode results. The older v3 synthesis command remains available for historical artifact regeneration, but the final Desktop artifact is built through the v4 path above.

N Self-Attack Ledgers

V3 base ledger.

Table 13: Fifty-round v3 self-attack ledger. Bounded rows are scoped limitations that the paper must not inflate.

Rnd.	Angle	Harsh attack	Defense or manuscript action	Status
1	novelty	This could be a generic candidate-budget paper.	Make codebook collisions, decode validity, hidden modes, rare tokens, and token-to-real calibration central.	pass
2	scope	The evidence is synthetic.	State controlled synthetic scope in abstract, limitations, audit, and checklist.	bounded
3	overclaim	The paper may imply tokenized world models are bad.	Explicitly say token models are not generally indicted.	pass
4	overclaim	The paper may imply larger candidate pools always hurt.	Explain the finite law as a tail amplifier, not an anti-N claim.	pass

Rnd.	Angle	Harsh attack	Defense or manuscript action	Status
5	overclaim	Repair could be sold as universal.	Limit repair to measurable aliasing or pilot labels.	pass
6	overclaim	No hardware validation.	No real-robot claim; final audit checks boundary language.	pass
7	overclaim	No external benchmark.	Call it controlled codebook-size stress only.	pass
8	overclaim	No state-of-the-art token model.	Frame as semi-learned mechanism artifact.	pass
9	mechanism	This is just a bad scorer.	The law shows top-score selection amplifies whatever the high-score tail contains.	pass
10	theory	The law is generic.	Concede generic law; tokenized population defines the contribution.	bounded
11	theory	Token scores can tie.	Use finite tie-aware score groups.	pass
12	theory	Monte Carlo validation may be noisy.	Report MAE and max error from exact-law validation.	pass
13	mechanism	Alias is vague.	Use codebook collision, alias probability, hidden-mode mismatch, and token-real gap.	pass
14	mechanism	Physical invalidity could be a separate bug.	Report physical-violation and decode-consistency diagnostics.	pass
15	mechanism	Hidden modes are artificial.	They isolate physical distinctions omitted by tokenization.	bounded
16	mechanism	Quantization error alone might explain failure.	Codebook sweep reports both collision and quantization.	pass
17	mechanism	Rare valid futures might be excluded.	Rare-mode reweighting and pilot calibration are reported separately.	pass
18	mechanism	Aggregate curves hide selected examples.	Tail autopsy compares raw, combined, and oracle selected candidates.	pass
19	scope	One codebook size is too narrow.	Use codebook bottleneck sweep across sizes.	pass
20	scope	Pool size may be too small.	Report full mode with 150 pools and pool size 64.	pass
21	leakage	Pilot calibration uses labels.	Separate token-to-real calibration from no-label diagnostics.	pass
22	leakage	Repairs use hand-designed diagnostics.	Treat diagnostics as controlled support, not deployment proof.	pass
23	claim strength	Repair is far from oracle.	Report combined and pilot oracle gaps explicitly.	pass
24	ablation	Codebook uncertainty alone is weak.	Report individual repair rows including codebook uncertainty.	pass
25	ablation	Decode consistency and physical filter may drive all gains.	Report decode, filter, rare, codebook, pilot, combined, oracle.	pass
26	baseline	Random selection could be enough.	Keep random in experiment code and discuss selector baselines if reported.	pass
27	calibration	Pilot calibration may overfit.	State pilot labels are controlled and require deployment refresh.	bounded
28	policy	Gate may be cosmetic.	Gate returns collect_pilot_labels with reason in summary.	pass
29	policy	Gate could be treated as deployment certification.	State gate is a controlled risk policy only.	pass
30	policy	Physical filters require domain checks.	Limit to tasks with meaningful validity checks.	pass
31	metric	Token-real gap may be confusing.	Define token score, real utility, token-real gap, alias and validity metrics.	pass
32	statistics	Intervals may be missing.	Use CI columns in metrics and describe them.	pass
33	statistics	Law only holds for fixed pools.	State fixed empirical pool assumption.	pass

Rnd.	Angle	Harsh attack	Defense or manuscript action	Status
34	statistics	Candidates may be correlated in real planners.	List correlated rollouts as limitation.	pass
35	learned evidence	Learned VQ is tiny.	Frame as CPU semi-learned mechanism artifact.	bounded
36	learned evidence	Transition model evidence is thin.	Report transition self-probability and codebook entropy.	pass
37	learned evidence	Decoder quality may dominate.	Use decode-consistency and quantization diagnostics.	pass
38	presentation	Six figures are too thin for v3.	Add v3 scorecard, repair, bottleneck, tail, VQ, attack figures.	pass
39	presentation	Claims need artifact traceability.	Add v3 scorecard table and artifact inventory.	pass
40	presentation	Current reviewer attacks are too short.	Add exactly 50 attack rounds.	pass
41	submission	11 pages is too short.	V3 build requires >=25 pages.	pass
42	workflow	Fresh agents may not find the repo.	V3 audit checks exact source-map row.	pass
43	workflow	Old v2 could remain visible.	V3 build removes old v2.	pass
44	workflow	Pushed branch might not be default.	Push final commit to main and verify remote SHA.	pass
45	reproducibility	Smoke run could overwrite full artifacts.	Avoid smoke after full; use cached synthesis and claim audit.	pass
46	reproducibility	Full experiments could be memory-heavy.	Use compact CSV/JSON, sequential runs, and cached synthesis.	pass
47	reproducibility	Repo and Desktop PDFs could diverge.	V3 audit checks repo/Desktop SHA match.	pass
48	reproducibility	Overclaims can creep into text.	V3 audit scans LaTeX plus base claim audit scans docs.	pass
49	novelty	It may look like another paper in the batch.	Token/codebook/decode vocabulary dominates all sections.	pass
50	framing	Limitations could be boilerplate.	Limitations name fixed pools, iid sampling, synthetic hidden modes, pilot labels, and no benchmarks.	pass

V4 final ledger. The v4 ledger extends the base attacks to citation surface, benchmark-boundary, protocol-freeze, rubric, source-firewall, and fresh-session reproducibility failures.

Table 14: Full 60-round v4 reviewer attack ledger.

Rnd.	Angle	Attack	Response	Status
1	novelty	This could be a generic candidate-budget paper.	Make codebook collisions, decode validity, hidden modes, rare tokens, and token-to-real calibration central.	PASS
2	scope	The evidence is synthetic.	State controlled synthetic scope in abstract, limitations, audit, and checklist.	PASS
3	overclaim	The paper may imply tokenized world models are bad.	Explicitly say token models are not generally indicted.	PASS
4	overclaim	The paper may imply larger candidate pools always hurt.	Explain the finite law as a tail amplifier, not an anti-N claim.	PASS
5	overclaim	Repair could be sold as universal.	Limit repair to measurable aliasing or pilot labels.	PASS
6	overclaim	No hardware validation.	No real-robot claim; final audit checks boundary language.	PASS
7	overclaim	No external benchmark.	Call it controlled codebook-size stress only.	PASS

Rnd.	Angle	Attack	Response	Status
8	overclaim	No state-of-the-art token model.	Frame as semi-learned mechanism artifact.	PASS
9	mechanism	This is just a bad scorer.	The law shows top-score selection amplifies whatever the high-score tail contains.	PASS
10	theory	The law is generic.	Concede generic law; tokenized population defines the contribution.	PASS
11	theory	Token scores can tie.	Use finite tie-aware score groups.	PASS
12	theory	Monte Carlo validation may be noisy.	Report MAE and max error from exact-law validation.	PASS
13	mechanism	Alias is vague.	Use codebook collision, alias probability, hidden-mode mismatch, and token-real gap.	PASS
14	mechanism	Physical invalidity could be a separate bug.	Report physical-violation and decode-consistency diagnostics.	PASS
15	mechanism	Hidden modes are artificial.	They isolate physical distinctions omitted by tokenization.	PASS
16	mechanism	Quantization error alone might explain failure.	Codebook sweep reports both collision and quantization.	PASS
17	mechanism	Rare valid futures might be excluded.	Rare-mode reweighting and pilot calibration are reported separately.	PASS
18	mechanism	Aggregate curves hide selected examples.	Tail autopsy compares raw, combined, and oracle selected candidates.	PASS
19	scope	One codebook size is too narrow.	Use codebook bottleneck sweep across sizes.	PASS
20	scope	Pool size may be too small.	Report full mode with 150 pools and pool size 64.	PASS
21	leakage	Pilot calibration uses labels.	Separate token-to-real calibration from no-label diagnostics.	PASS
22	leakage	Repairs use hand-designed diagnostics.	Treat diagnostics as controlled support, not deployment proof.	PASS
23	claim strength	Repair is far from oracle.	Report combined and pilot oracle gaps explicitly.	PASS
24	ablation	Codebook uncertainty alone is weak.	Report individual repair rows including codebook uncertainty.	PASS
25	ablation	Decode consistency and physical filter may drive all gains.	Report decode, filter, rare, codebook, pilot, combined, oracle.	PASS
26	baseline	Random selection could be enough.	Keep random in experiment code and discuss selector baselines if reported.	PASS
27	calibration	Pilot calibration may overfit.	State pilot labels are controlled and require deployment refresh.	PASS
28	policy	Gate may be cosmetic.	Gate returns collect_pilot_labels with reason in summary.	PASS
29	policy	Gate could be treated as deployment certification.	State gate is a controlled risk policy only.	PASS
30	policy	Physical filters require domain checks.	Limit to tasks with meaningful validity checks.	PASS
31	metric	Token-real gap may be confusing.	Define token score, real utility, token-real gap, alias and validity metrics.	PASS
32	statistics	Intervals may be missing.	Use CI columns in metrics and describe them.	PASS
33	statistics	Law only holds for fixed pools.	State fixed empirical pool assumption.	PASS
34	statistics	Candidates may be correlated in real planners.	List correlated rollouts as limitation.	PASS
35	learned evidence	Learned VQ is tiny.	Frame as CPU semi-learned mechanism artifact.	PASS
36	learned evidence	Transition model evidence is thin.	Report transition self-probability and codebook entropy.	PASS
37	learned evidence	Decoder quality may dominate.	Use decode-consistency and quantization diagnostics.	PASS

Rnd.	Angle	Attack	Response	Status
38	presentation	Six figures are too thin for v3.	Add v3 scorecard, repair, bottleneck, tail, VQ, attack figures.	PASS
39	presentation	Claims need artifact traceability.	Add v3 scorecard table and artifact inventory.	PASS
40	presentation	Current reviewer attacks are too short.	Add exactly 50 attack rounds.	PASS
41	submission	11 pages is too short.	V3 build requires >=25 pages.	PASS
42	workflow	Fresh agents may not find the repo.	V3 audit checks exact source-map row.	PASS
43	workflow	Old v2 could remain visible.	V3 build removes old v2.	PASS
44	workflow	Pushed branch might not be default.	Push final commit to main and verify remote SHA.	PASS
45	reproducibility	Smoke run could overwrite full artifacts.	Avoid smoke after full; use cached synthesis and claim audit.	PASS
46	reproducibility	Full experiments could be memory-heavy.	Use compact CSV/JSON, sequential runs, and cached synthesis.	PASS
47	reproducibility	Repo and Desktop PDFs could diverge.	V3 audit checks repo/Desktop SHA match.	PASS
48	reproducibility	Overclaims can creep into text.	V3 audit scans LaTeX plus base claim audit scans docs.	PASS
49	novelty	It may look like another paper in the batch.	Token/codebook/decode vocabulary dominates all sections.	PASS
50	framing	Limitations could be boilerplate.	Limitations name fixed pools, iid sampling, synthetic hidden modes, pilot labels, and no benchmarks.	PASS
51	citation	The related work could be too thin for ICLR.	Keep explicit citations for VQ, world models, reranking, calibration, and safety filters.	PASS
52	benchmark	The paper has no external benchmark.	Frame the codebook sweep and token stress suite as controlled mechanism stress, not external validation.	PASS
53	benchmark	The codebook sweep may be mistaken for broad validation.	State that it is codebook-size stress inside the controlled generator.	PASS
54	novelty	The finite law could look like a generic wrapper.	Make token codebooks, collisions, decode validity, and physical aliasing the paper identity.	PASS
55	baseline	A simple physical filter might be enough.	Keep physical filter, decode consistency, rare-mode, pilot calibration, combined repair, and oracle rows separated.	PASS
56	calibration	Pilot labels may be an unfair hidden oracle.	Label token-to-real calibration as label-spending evidence and keep no-label diagnostics separate.	PASS
57	protocol	The final story might keep adapting after failures.	Freeze code, seeds, candidate pools, budgets, selectors, metrics, thresholds, and claim gates before reporting.	PASS
58	protocol	Stress tests may be cherry-picked.	Use stress failures as adversarial teachers during development; final evidence is measurement-only.	PASS
59	rubric	The manuscript may pass local checks but miss review criteria.	Generate novelty, rigor, baseline, statistics, reproducibility, and scope-control rubric gates.	PASS
60	reproducibility	A fresh session might not find the final artifact.	Build tokenized world model-v4.pdf, update Desktop source map, write manifest, and verify GitHub SHA.	PASS

O FINAL V4 ACCEPTANCE CHECKLIST

- The final PDF must have at least 25 pages.

- The repo final PDF and Desktop PDF must match by SHA-256.
- The Desktop file must be `tokenized world model-v4.pdf`.
- The old Desktop v3 and v2 files must be absent.
- `PAPER_SOURCE_MAP.md` must map the v4 Desktop filename to:

  ```
  C:/Users/wangz/tokenized world model
  Jason-Wang313/tokenized-world-model
  ```
- The v4 self-attack ledger must contain exactly 60 rows.
- The v4 protocol gates must pass 12/12, and the rubric map must pass 7/7.
- The manuscript must not contain unsupported claims about hardware deployment, external benchmark coverage, leaderboard performance, universal high-budget harm, or universal repair.
- The final commit must be pushed to GitHub `main`, and the remote SHA must be verified.